Subject: ASD&D
Semester: V
Class: TE

Batch: T11
Name: Irisha Deshpande
Roll No: 2401022

Experiment 5

Aim: To automate application deployment using Github/Jenkins.
Lab Outcome: Mapped LO3
LO3: Build and manage automated CI/CD pipelines for application development, testing, packaging and deployment.

Theory:

Continuous Integration (CI) and Continuous Deployment (CD) are important DevOps practices that help connect software development with deployment and operational activities.

- **Continuous Integration (CI):** CI is a development approach in which developers frequently combine their code changes into a shared repository. These changes are then automatically built and tested. This helps identify errors early, improve software quality, and reduce the effort required for manual validation.
- **Continuous Deployment (CD):** CD is the automated process of taking verified changes, preparing them for release, and deploying them to a production or staging environment without requiring manual deployment steps.
- **Jenkins & Declarative Pipelines:** Jenkins is an open-source automation tool used to automate processes such as building, testing, and deploying applications. A Declarative Pipeline uses a structured Groovy-based syntax to define the pipeline's stages, steps, and execution environment. These instructions are stored in a Jenkinsfile, making the pipeline version-controlled, repeatable, and easier to maintain.

Step 1: Jenkins Server Initialization and Access Setup

- Jenkins operates as a background service and requires a compatible Java Development Kit (JDK 21) to run.
- The Jenkins web interface can be accessed through http://localhost:8080.

- During the initial setup, an administrator account is created. This account is used to manage Jenkins permissions, jobs, configurations, and other server activities.

Step 2: Baseline Pipeline Validation

- A new Jenkins project named **GitHub-Jenkins-Demo** is created using the Pipeline job type.
- Before connecting Jenkins to GitHub, a Declarative Pipeline is entered directly into the Jenkins job configuration to verify that the Jenkins execution environment works correctly.
- The pipeline uses the agent any directive and contains three stages: **Build, Test, and Deploy**. Each stage initially performs a simple echo operation.
- Running **Build #1** allows Jenkins to execute these stages sequentially on an available node and display the results in the console. A successful execution ends with a **SUCCESS** status.

Step 3: GitHub and Pipeline-as-Code

- A GitHub repository named **exp5_asd** is created to store the application source code and Jenkins automation configuration.
- An index.html file is added to serve as the frontend application.
- A Jenkinsfile is created in the root of the repository using Declarative Pipeline syntax.
- The Deploy stage contains commands that check or create the deployment directory and copy index.html from the Jenkins workspace to the specified deployment location.

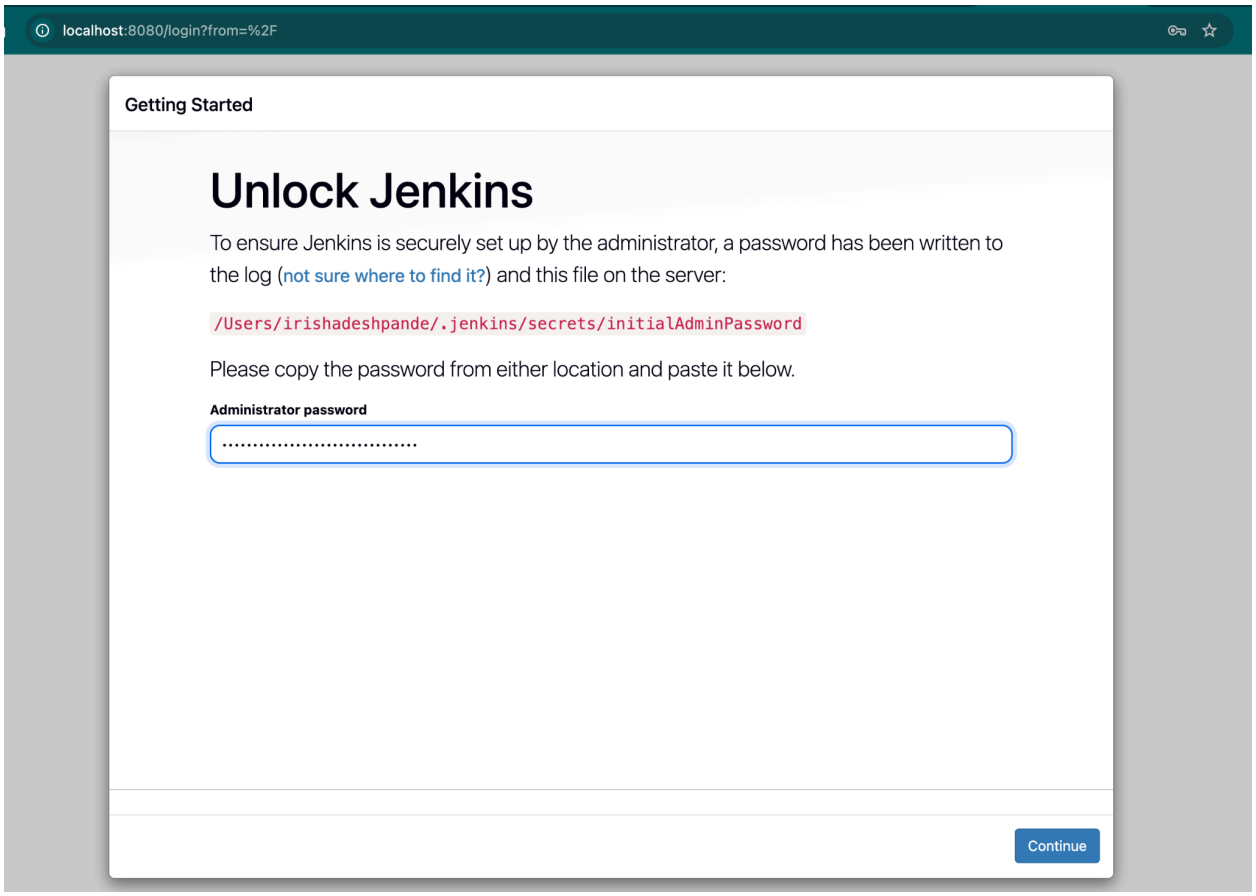
Step 4: Connecting Jenkins to GitHub

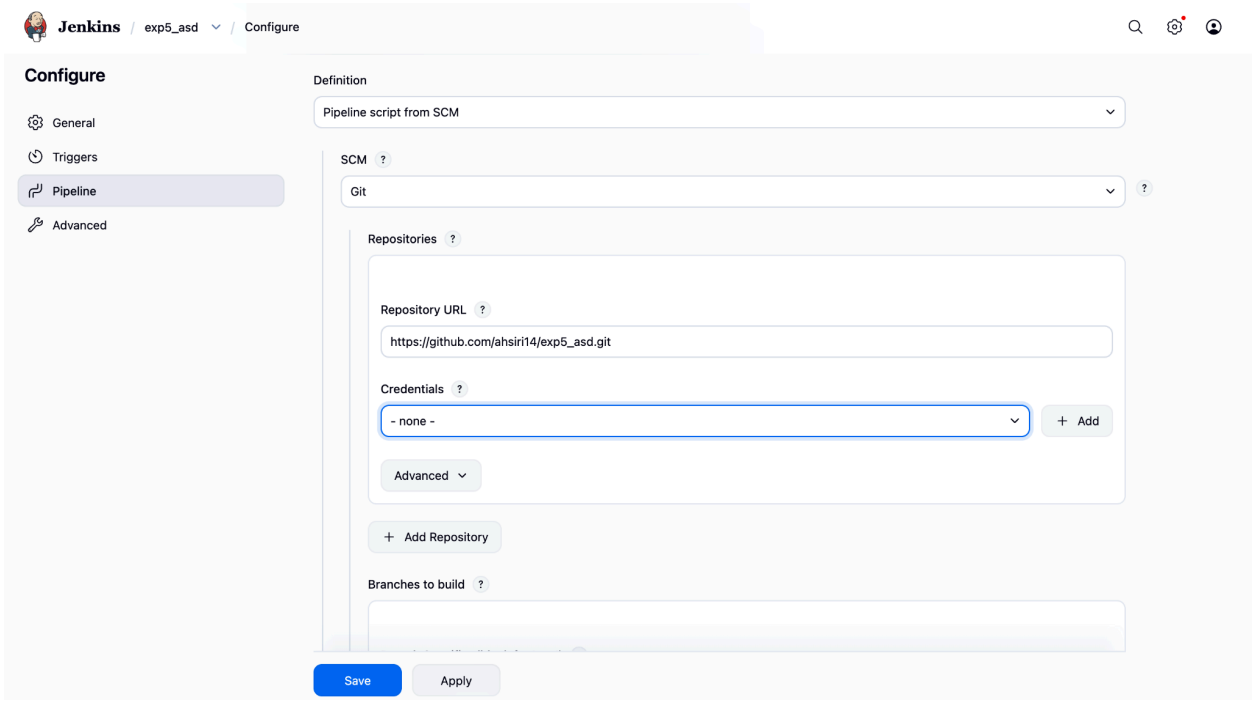
- The Jenkins project is changed from using an inline Pipeline script to **Pipeline script from SCM**.
- Git is selected as the source-code management system.
- The GitHub repository's HTTPS URL is entered, the branch is configured as */main, and the script path is set to Jenkinsfile.
- When **Build #2** is executed, Jenkins retrieves the latest code from GitHub, checks out the selected branch, reads the Jenkinsfile, and executes its pipeline stages.
- The deployed index.html file is then opened in a browser to confirm that the initial application has been successfully deployed.

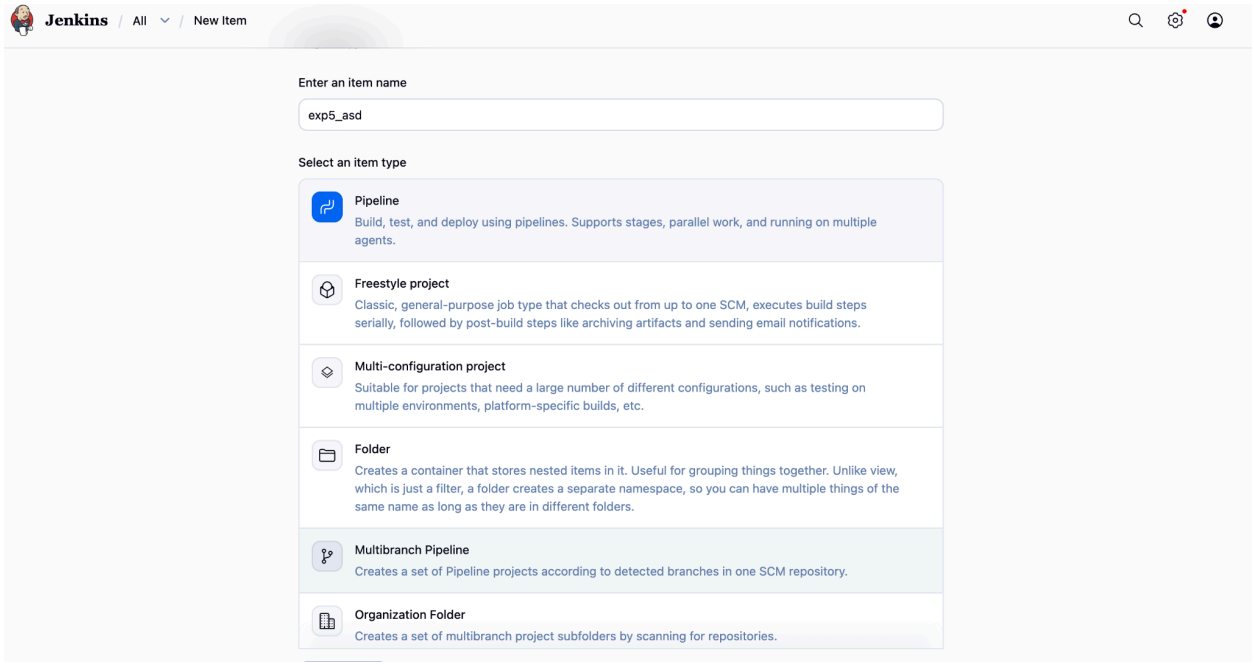
Step 5: CI/CD Verification

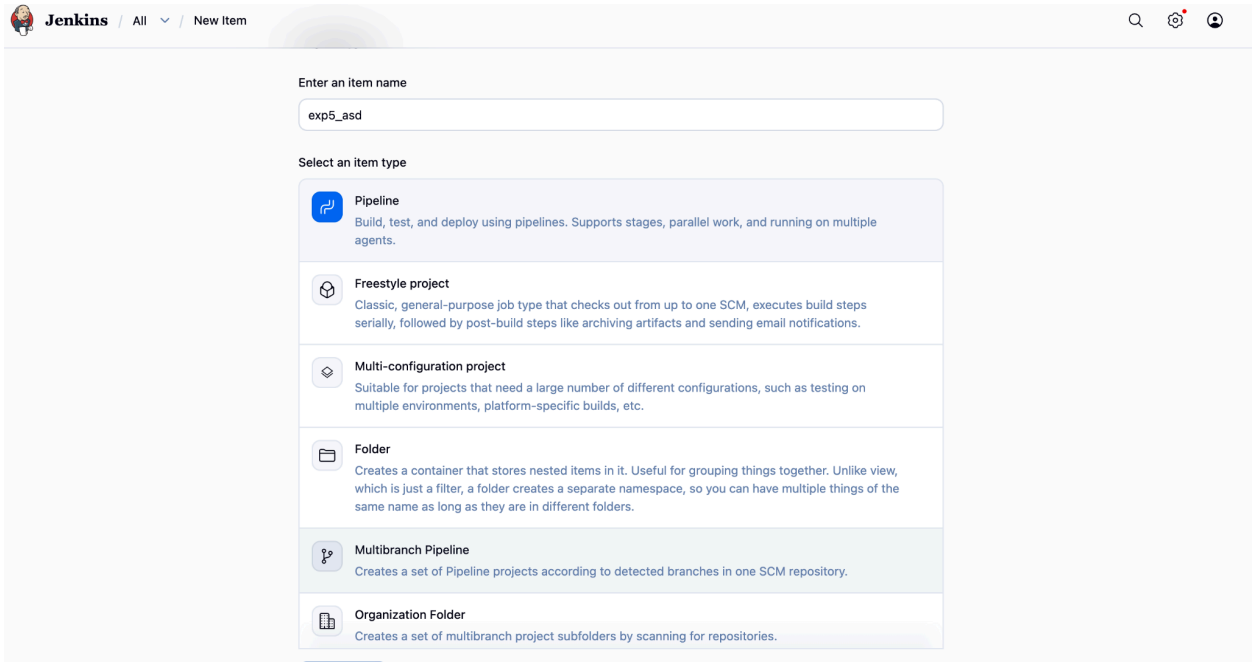
- To demonstrate continuous deployment, index.html is modified directly in GitHub. CSS styling is added and the page text is changed to indicate that **Version 2.0** has been deployed through the CI/CD pipeline.
- When **Build #3** is executed, Jenkins retrieves the updated code from GitHub and runs the Build, Test, and Deploy stages again.
- The new index.html replaces the previous version in the deployment directory.
- Refreshing the webpage displays the updated design and **Version 2.0** message, demonstrating the complete flow from a code change in GitHub through Jenkins automation to deployment.

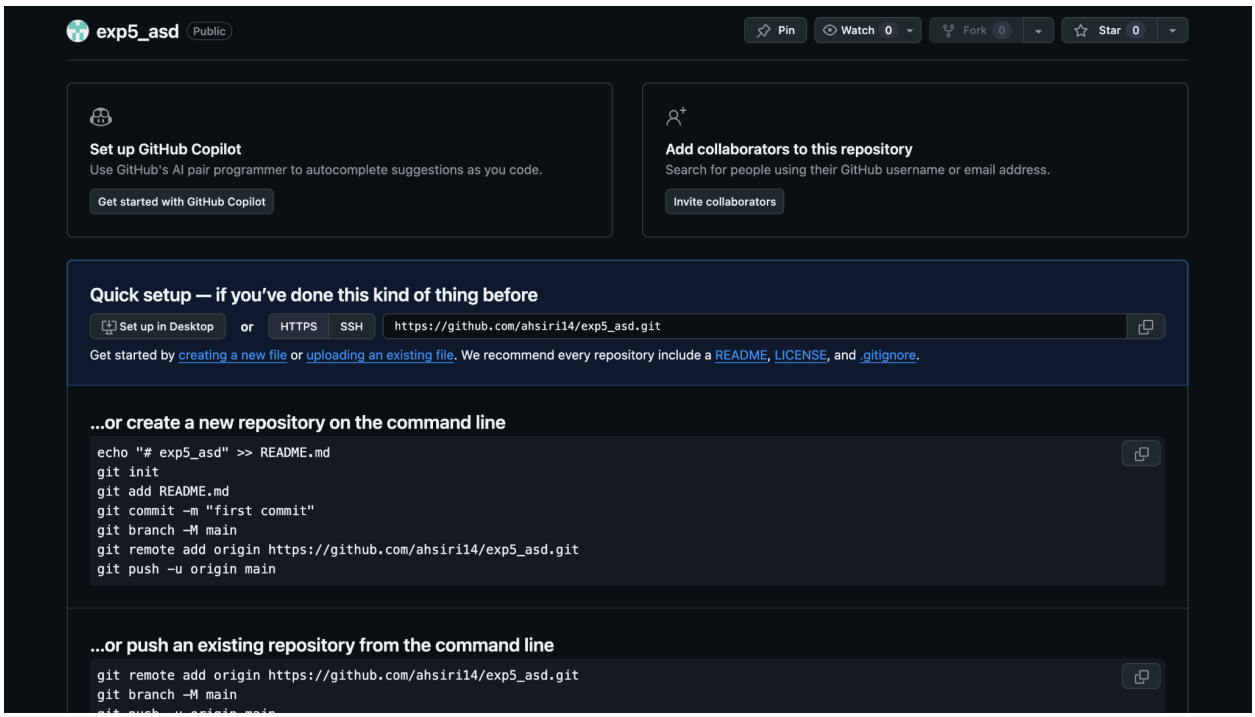
Execution:

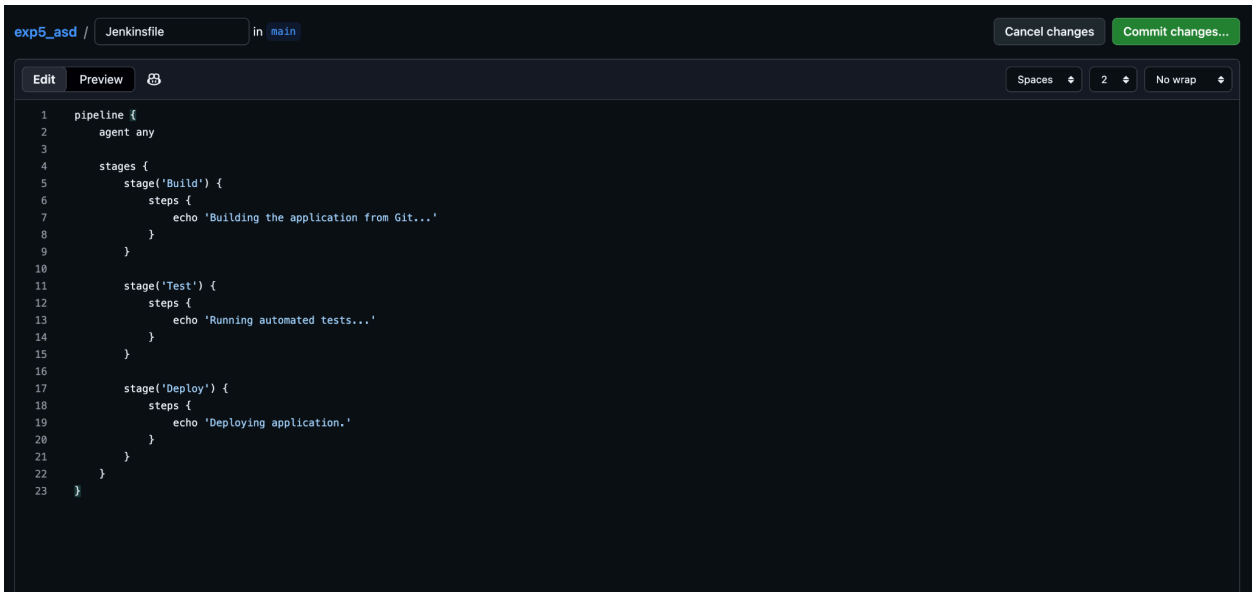

localhost:8080/login?from=%2F
Getting Started
Unlock Jenkins
To ensure Jenkins is securely set up by the administrator, a password has been written to the log (not sure where to find it?) and this file on the server:
/Users/irishadeshpande/.jenkins/secrets/initialAdminPassword
Please copy the password from either location and paste it below.
Administrator password
Continue

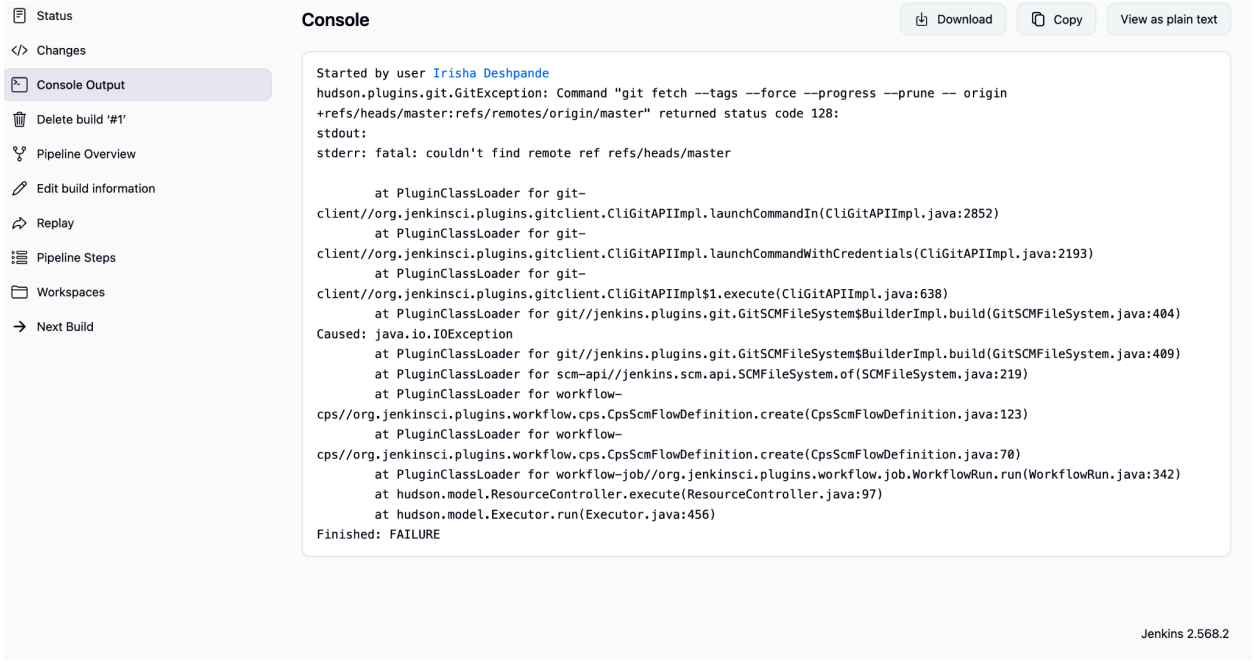

Jenkins / exp5_asd / Configure
Configure
General
Triggers
Pipeline
Advanced
Definition
Pipeline script from SCM
SCM
Git
Repositories
Repository URL
https://github.com/ahsiri14/exp5_asd.git
Credentials
- none -
Add
Advanced
Add Repository
Branches to build
Save
Apply


Jenkins / All / New Item
Enter an item name
exp5_asd
Select an item type
Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
Folder
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.
Multibranch Pipeline
Creates a set of Pipeline projects according to detected branches in one SCM repository.
Organization Folder
Creates a set of multibranch project subfolders by scanning for repositories.


Jenkins / All / New Item
Enter an item name
exp5_asd
Select an item type
Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
Folder
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.
Multibranch Pipeline
Creates a set of Pipeline projects according to detected branches in one SCM repository.
Organization Folder
Creates a set of multibranch project subfolders by scanning for repositories.


exp5_asd
Public
Pin
Watch 0
Fork 0
Star 0
Set up GitHub Copilot
Use GitHub's AI pair programmer to autocomplete suggestions as you code.
Get started with GitHub Copilot
Add collaborators to this repository
Search for people using their GitHub username or email address.
Invite collaborators
Quick setup — if you've done this kind of thing before
Set up in Desktop
or
HTTPS
SSH
https://github.com/ahsiri14/exp5_asd.git
Get started by creating a new file or uploading an existing file. We recommend every repository include a README, LICENSE, and .gitignore.
...or create a new repository on the command line
echo "# exp5_asd" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/ahsiri14/exp5_asd.git
git push -u origin main
...or push an existing repository from the command line
git remote add origin https://github.com/ahsiri14/exp5_asd.git
git branch -M main


exp5_asd / Jenkinsfile in main
Cancel changes
Commit changes...
Edit
Preview
Spaces
2
No wrap
pipeline {
agent any
stages {
stage('Build') {
steps {
echo 'Building the application from Git...'
}
}
stage('Test') {
steps {
echo 'Running automated tests...'
}
}
stage('Deploy') {
steps {
echo 'Deploying application.'
}
}
}
}


Status
Changes
Console Output
Delete build '#1'
Pipeline Overview
Edit build information
Replay
Pipeline Steps
Workspaces
Next Build
Console
Download
Copy
View as plain text
Started by user Irisha Deshpande
hudson.plugins.git.GitException: Command "git fetch --tags --force --progress --prune -- origin
+refs/heads/master:refs/remotes/origin/master" returned status code 128:
stdout:
stderr: fatal: couldn't find remote ref refs/heads/master
at PluginClassLoader for git-
client//org.jenkinsci.plugins.gitclient.CliGitAPIImpl.launchCommandIn(CliGitAPIImpl.java:2852)
at PluginClassLoader for git-
client//org.jenkinsci.plugins.gitclient.CliGitAPIImpl.launchCommandWithCredentials(CliGitAPIImpl.java:2193)
at PluginClassLoader for git-
client//org.jenkinsci.plugins.gitclient.CliGitAPIImpl$1.execute(CliGitAPIImpl.java:638)
at PluginClassLoader for git//jenkins.plugins.git.GitSCMFileSystem$BuilderImpl.build(GitSCMFileSystem.java:404)
Caused: java.io.IOException
at PluginClassLoader for git//jenkins.plugins.git.GitSCMFileSystem$BuilderImpl.build(GitSCMFileSystem.java:409)
at PluginClassLoader for scm-api//jenkins.scm.api.SCMFileSystem.of(SCMFileSystem.java:219)
at PluginClassLoader for workflow-
cps//org.jenkinsci.plugins.workflow.cps.CpsScmFlowDefinition.create(CpsScmFlowDefinition.java:123)
at PluginClassLoader for workflow-
cps//org.jenkinsci.plugins.workflow.cps.CpsScmFlowDefinition.create(CpsScmFlowDefinition.java:70)
at PluginClassLoader for workflow-job//org.jenkinsci.plugins.workflow.job.WorkflowRun.run(WorkflowRun.java:342)
at hudson.model.ResourceController.execute(ResourceController.java:97)
at hudson.model.Executor.run(Executor.java:456)
Finished: FAILURE
Jenkins 2.568.2

```
 > git config core.sparsecheckout # timeout=10
 > git checkout -f e1115a521dcf0055f4b1f802ecba2f61da8c068f # timeout=10
Commit message: "Create Jenkinsfile"
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Building the application from Git...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Running automated tests...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploying application.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

ahsiri14 Create index.html

Code | Blame | 10 lines (10 loc) · 188 Bytes

```
1   <!DOCTYPE html>
2   <html>
3   <head>
4       <title>Jenkins Demo App</title>
5   </head>
6   <body>
7       <h1>Hello from Jenkins!</h1>
8       <p>Application deployed using GitHub and Jenkins.</p>
9   </body>
10  </html>
```



Hello from Jenkins!

Application deployed using GitHub and Jenkins.

Conclusion:

The experiment successfully demonstrated the use of Jenkins and GitHub to implement a basic CI/CD pipeline. A Declarative Pipeline was created using a Jenkinsfile with Build, Test, and Deploy stages. Jenkins was connected to the GitHub repository and successfully retrieved the application source code. The deployment process was automated so that changes to index.html could be built, tested, and deployed through Jenkins. The updated Version 2.0 application was then verified, demonstrating how CI/CD can automate the delivery of application changes from source control to the deployment environment.